

Arbitrary Precision Arithmetic Library: Project Report

Software Development Fundamentals (CS1023)

Shiva Ganesh Adepu
ID: CS24BTECH11003

May 2025

Contents

1	Introduction	2
2	Design	2
3	Logic of Classes	3
3.1	AInteger Class Logic	3
3.1.1	Arithmetic Operations Logic	4
3.2	AFloat Class Logic	4
3.2.1	Arithmetic Operations Logic	5
4	Usage Instructions	6
4.1	As a Command-Line Application	6
4.2	As a Library	6
5	Limitations	6
6	Verification Approach	7
7	Key Learnings	7

1 Introduction

This report outlines the development of an arbitrary precision arithmetic library in Java. The library, implemented in the `arbitraryarithmetic` package, supports infinite-precision arithmetic operations for both integers and floating-point numbers through two classes: `AInteger` and `AFloat`. The project also includes a command-line interface via the `MyInfArith` class and a Python script for compilation and execution.

The library supports addition, subtraction, multiplication, and division operations without round-off errors, adhering to the project requirement of preserving complete numerical information. Decimal precision for floating-point operations is truncated at 30 digits, following Java's `BigDecimal` class behavior.

2 Design

The library is structured around two core classes within the `arbitraryarithmetic` package: `AInteger` for integer arithmetic and `AFloat` for floating-point arithmetic. Key design decisions include:

- **String-based Representation:** Both classes internally represent numbers as strings to handle arbitrary precision, avoiding limitations of Java's primitive types.
- **Sign Handling:** The `AInteger` class uses a boolean `isPositive` to track the sign, while `AFloat` relies on `AInteger` for sign management.
- **Decimal Tracking in AFloat:** The `AFloat` class tracks decimal places using `NumOfDigitsAfterDecimal`, enabling precise floating-point operations.
- **Operator Overloading Simulation:** Since Java does not support operator overloading, methods like `add`, `subtract`, `multiply`, and `divide` are implemented to simulate operator behavior.
- **Reusability:** Helper methods like `removeLeadingZeros` and `compareStrings` in `AInteger` are reused across operations to ensure consistency.

The class structure and relationships are illustrated in the UML diagram (see Figure 1).

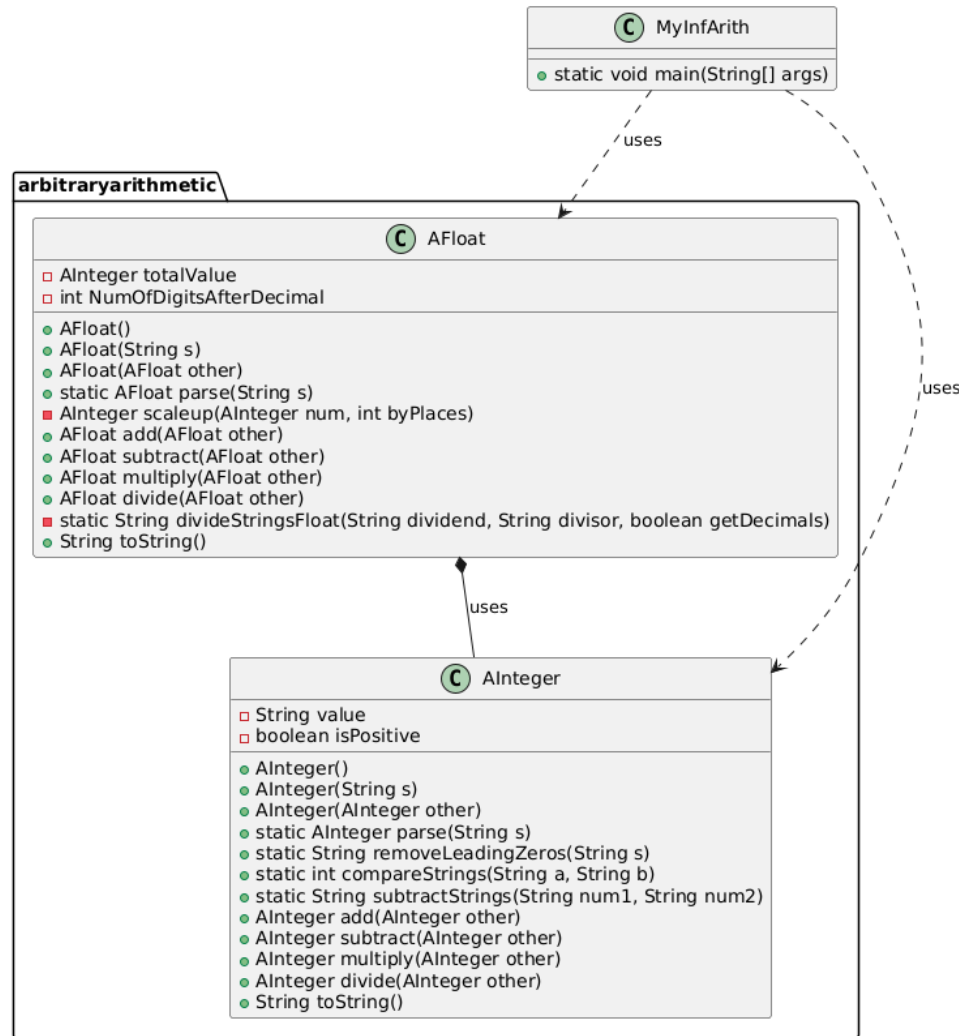


Figure 1: UML Class Diagram for the Arbitrary Precision Arithmetic Library

3 Logic of Classes

This section details the core logic behind the `AIInteger` and `AFloat` classes, focusing on how they handle arbitrary precision arithmetic and the role of method overriding.

3.1 AIInteger Class Logic

The `AIInteger` class is designed to handle arbitrary precision integer arithmetic. Numbers are stored as strings in the `value` field, with a boolean `isPositive` to track the sign. The core logic revolves around string manipulation to perform arithmetic operations:

- **Constructor and Parsing:** The class provides constructors to initialize an `AIInteger` from a string or another `AIInteger`. The `parse` method converts a string input into an `AIInteger`, handling signs and leading zeros using `removeLeadingZeros`.

- **Arithmetic Operations:** Methods like `add`, `subtract`, `multiply`, and `divide` implement arithmetic by manipulating digit strings. For example, `addStrings` and `subtractStrings` perform digit-by-digit addition and subtraction, managing carries and borrows, while `multiplyStrings` and `divideStrings` use schoolbook algorithms for multiplication and division.
- **Utility Methods:** Helper methods like `compareStrings` compare two number strings by length and digit values, ensuring correct ordering for subtraction and division.
- **Method Overriding:** The `toString` method is overridden (using Java's `@Override` annotation) to provide a string representation of the number, ensuring the sign is correctly displayed (e.g., adding a negative sign if `isPositive` is false).

The `@Override` annotation in `toString` indicates that the method overrides the default `toString` implementation from Java's `Object` class, ensuring proper string conversion for `BigInteger` objects.

3.1.1 Arithmetic Operations Logic

The following details the logic for each arithmetic operation in `BigInteger`:

- **Addition:** The `add` method first checks the signs of the operands. If both are positive or both negative, it calls `addStrings` to perform digit-by-digit addition, handling carries (e.g., for `"123" + "456"`, it computes each column: $3+6=9$, $2+5=7$, $1+4=5$, resulting in `"579"`). If signs differ, it converts addition into subtraction by comparing magnitudes with `compareStrings` and calling `subtractStrings`.
- **Subtraction:** The `subtract` method also considers signs. If both operands have the same sign, it uses `subtractStrings` for digit-by-digit subtraction with borrowing (e.g., `"456" - "123"` computes $6-3=3$, $5-2=3$, $4-1=3$, yielding `"333"`). If signs differ, it converts subtraction into addition. The `compareStrings` method ensures the larger number is subtracted from the smaller one, adjusting the sign of the result.
- **Multiplication:** The `multiply` method implements a schoolbook multiplication algorithm via `multiplyStrings`. It multiplies each digit of one number by each digit of the other, tracking carries and summing partial products (e.g., `"12" × "13"` computes $12 \times 3 = 36$ and $12 \times 1 = 12$, then adds with proper place alignment: $36 + 120 = 156$). Signs are handled by setting the result's `isPositive` based on the XOR of the operands' signs.
- **Division:** The `divide` method uses `divideStrings`, implementing long division. It repeatedly subtracts the divisor from the dividend, counting how many times it fits (e.g., `"100" ÷ "4"` subtracts 4 from 100, 25 times, yielding `"25"`). It handles division by zero by throwing an exception and sets the sign of the result based on the operands' signs.

3.2 AFloat Class Logic

The `AFloat` class extends the functionality of `BigInteger` to support floating-point arithmetic. It reuses much of `BigInteger`'s logic, with additional handling for decimal points:

- **Structure and Reuse:** `AFloat` stores its value as an `AInteger` in the `totalValue` field, with an integer `NumOfDigitsAfterDecimal` to track the number of digits after the decimal point. This allows `AFloat` to leverage `AInteger`'s integer arithmetic logic by converting floating-point numbers into integers (e.g., 12.34 is stored as 1234 with `NumOfDigitsAfterDecimal = 2`).
- **Decimal Handling:** Arithmetic operations in `AFloat` first align the decimal points of operands by scaling them (using `scaleup`), then perform the operation using `AInteger` methods, and finally adjust the result's decimal position. For example, in `add`, two `AFloat` numbers are scaled to have the same number of decimal places, added as `AIntegers`, and the result is converted back to an `AFloat`.
- **Division Precision:** The `divideStringsFloat` method ensures floating-point division results are truncated at 30 decimal places, as per project requirements, while reusing `AInteger`'s division logic for the core computation.
- **Method Overriding:** Similar to `AInteger`, `AFloat` overrides the `toString` method using the `@Override` annotation. This method converts the `totalValue` back into a floating-point string by inserting the decimal point at the correct position based on `NumOfDigitsAfterDecimal`.

The use of `@Override` in both classes ensures that the `toString` method adheres to Java's object-oriented principles, providing a customized string representation for each class while maintaining compatibility with the `Object` superclass.

3.2.1 Arithmetic Operations Logic

The following details the logic for each arithmetic operation in `AFloat`, building on `AInteger`'s logic:

- **Addition:** The `add` method aligns the decimal points of the two `AFloat` numbers by scaling them with `scaleup` (e.g., for $12.34 + 56.7$, it converts to 1234 and 5670 with `NumOfDigitsAfterDecimal = 2`). It then uses `AInteger`'s `add` method to compute the sum ($1234 + 5670 = 6904$), adjusts the decimal position in the result (69.04), and returns a new `AFloat`.
- **Subtraction:** The `subtract` method follows a similar process, aligning decimals (e.g., $56.7 - 12.34$ becomes $5670 - 1234$), uses `AInteger`'s `subtract` ($5670 - 1234 = 4436$), and adjusts the result's decimal position (44.36). Sign handling is inherited from `AInteger`.
- **Multiplication:** The `multiply` method first multiplies the `totalValue` of both `AFloat` numbers using `AInteger`'s `multiply` (e.g., 12.34×56.78 becomes $1234 \times 5678 = 7006652$). It then calculates the new `NumOfDigitsAfterDecimal` by adding the decimal places of the operands ($2 + 2 = 4$), resulting in 700.6652.
- **Division:** The `divide` method uses `divideStringsFloat`, which scales the dividend to increase precision (e.g., for $12.34 \div 56.78$, it scales 1234 to 123400... with 30 extra digits), performs division using `AInteger`'s logic (yielding a quotient with extra precision), and truncates the result at 30 decimal places (e.g., 0.217330...).

4 Usage Instructions

The library can be used in two modes: as a command-line application or as a reusable library.

4.1 As a Command-Line Application

The `MyInfArith` class provides a command-line interface. It accepts four arguments:

- `<int/float>`: Specifies the type of operation (`int` for integers, `float` for floating-point).
- `<add/sub/mul/div>`: Specifies the operation.
- `First operand`: The first number as a string.
- `Second operand`: The second number as a string.

Example usage:

```
java MyInfArith int add 23650078224912949497310933240250 4293978326246711
```

Output:

```
66589861487380063295697317641748
```

A Python script (`PythonScript.py`) is provided to compile and run the project. Run it as follows:

```
python3 PythonScript.py
```

4.2 As a Library

The `arithmetic.jar` file in the `arbitraryarithmetic` directory can be included in other Java projects. Import the classes as follows:

```
import arbitraryarithmetic.AInteger;  
import arbitraryarithmetic.AFloat;
```

Example usage in code:

```
AInteger a = new AInteger("123456789");  
AInteger b = new AInteger("987654321");  
AInteger sum = a.add(b);  
System.out.println(sum); // Outputs: 1111111110
```

5 Limitations

The library has the following limitations:

- **Performance:** String-based arithmetic operations are slower than native types, especially for large numbers.
- **Decimal Precision:** Floating-point division is capped at 30 decimal places, which may not suffice for applications requiring higher precision.

- **Error Handling:** Division by zero is handled, but other edge cases (e.g., extremely large inputs) may lead to memory issues.
- **Lack of Optimization:** Operations like multiplication and division could be optimized using advanced algorithms (e.g., Karatsuba multiplication).

6 Verification Approach

The library was tested across multiple stages:

- **Unit Testing:** Individual methods in `AInteger` and `AFloat` were tested with small inputs (e.g., `"123" + "456"`).
- **Test Cases:** The provided test cases were executed via `MyInfArith`, covering integer and floating-point operations (e.g., `java MyInfArith float div 87927263652830605798493835253617089647454998358`).
- **Edge Cases:** Tested with zero, negative numbers, and large inputs to ensure robustness.

7 Key Learnings

This project provided valuable insights:

- **Arbitrary Precision:** Implementing arithmetic without round-off errors required careful string manipulation and decimal tracking.
- **Java Limitations:** Working around Java's lack of operator overloading necessitated method-based operation design.
- **Project Management:** Using Git for version control and tagging stages improved collaboration and tracking.
- **Documentation:** Writing detailed comments and a LaTeX report enhanced the project's clarity and usability.